

Assignment Report

On

Analysis of Sorting Algorithms

Submitted by

263CS002 Mukul Rajput

262IS031 Sujal Gupta

Dept. of Computer Science,
NITK, Surathkal



Department of Information Technology
National Institute of Technology Karnataka,
Surathkal

Contents

Introduction	3
1 Comparison of Sorting Algorithms	4
1.1 Bubble Sort	4
1.2 Insertion Sort	4
1.3 Merge Sort	5
1.4 Quick Sort	5
1.5 Heap Sort	5
1.6 Radix Sort	5
2 Data Generation and Experimental Setup	5
2.1 Choice of Test Data	6
2.2 Experimental Setup	6
2.3 Stabilizing Measurements	7
3 Comparison of Quick Sort Versions	7
3.1 Best Case Running Time	7
3.2 Worst Case Running Time	8
3.3 Average Case Running Time	9
3.4 Conclusion	10
4 Detailed Analysis of Individual Sorting Algorithms	10
4.1 Bubble Sort	10
4.2 Insertion Sort	11
4.3 Merge Sort	12
4.4 Quick Sort	12
4.5 Heap Sort	13
4.6 Radix Sort	14
5 Comparison of Sorting Algorithms	14
5.1 Best Case Running Time	15
5.2 Worst Case Running Time	16
5.3 Average Case Running Time	17

5.4	Conclusion	17
6	Correlation of Comparisons with Execution Time	18
6.1	Insertion Sort	18
6.2	Bubble Sort (Optimized)	18
6.3	Heap Sort	19
6.4	Merge Sort	19
6.5	Quick Sort (Median-of-Three Pivot)	20
6.6	Radix Sort	21
6.7	Conclusion	21
7	Execution Time Analysis	21
7.1	Bubble Sort	22
7.2	Insertion Sort	22
7.3	Merge Sort	22
7.4	Heap Sort	22
7.5	Radix Sort	23
7.6	Quick Sort (Median-of-Three Pivot)	23
7.7	Quick Sort: Comparison of the Three Versions	23
	Overall Conclusion	24

Abstract

This report presents an experimental study of six fundamental sorting algorithms: Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, and Radix Sort. Each algorithm was implemented in C++ and timed on three kinds of input: randomly generated, increasing, and decreasing integer arrays. Input sizes range from 10,000 to 500,000 elements wherever measurements are available. Three versions of Quick Sort, which use the first element, a random element, and the median of three elements as the pivot, are compared separately to show how pivot selection affects partition quality. In addition to execution time, the number of comparisons performed was recorded, so that the relationship between comparison count and running time could be examined.

The results show that asymptotic labels alone do not decide which algorithm is fastest. Quick Sort with a first-element pivot is among the fastest methods on random data but takes more than 61 seconds on 500,000 decreasing elements. The median-of-three implementation used here shows a similar quadratic collapse on decreasing input, even though the strategy is normally considered safe. Merge Sort and Heap Sort grow smoothly regardless of input order, random-pivot Quick Sort is the most robust of the three Quick Sort variants, and Radix Sort, which uses no comparisons, is the fastest method on random data at large sizes.

Introduction

Sorting is one of the most common operations in computing. It underlies searching, database indexing, scheduling, and many other algorithms, and it is a standard vehicle for teaching algorithm analysis. Textbook analysis describes the running time of a sorting algorithm with asymptotic notation such as $O(n^2)$ or $O(n \log n)$. Such bounds describe how the cost grows with input size, but they hide constant factors, memory behavior, and the effect of the input order. In practice these hidden factors can determine which algorithm is the better choice for a given task.

The aim of this assignment is to compare the theoretical expectations for six classic sorting algorithms with what is actually measured when they are implemented and run. The study has the following objectives:

- to measure the running time of each algorithm on random, increasing, and decreasing inputs of different sizes;
- to compare three pivot-selection strategies for Quick Sort and see how sensitive each is to the input order;
- to compare all algorithms in best-case, worst-case, and average-case settings;
- to examine how strongly the number of comparisons explains the observed execution time; and
- to report the measured averages in tables so that every conclusion can be traced back to recorded data.

The rest of the report is organized as follows. Section 1 summarizes the algorithms and their theoretical behavior. Section 2 describes the test data and the experimental procedure. Section 3 compares the three Quick Sort variants. Section 4 analyzes each algorithm individually, and Section 5 compares the algorithms with one another. Section 6 relates comparison counts to execution time, and Section 7 lists the complete tables of average execution times. The report ends with overall conclusions and a discussion of limitations.

1 Comparison of Sorting Algorithms

This section gives a brief overview of the six algorithms and the principle behind each. The table below summarizes the standard theoretical running times against which the experimental results are later compared.

Algorithm	Best case	Average case	Worst case	Extra space
Bubble Sort (optimized)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Radix Sort	$O(d(n + k))$	$O(d(n + k))$	$O(d(n + k))$	$O(n + k)$

Theoretical time and space complexity of the algorithms studied (d is the number of digit positions and k the size of the digit alphabet).

1.1 Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them whenever they are in the wrong order. After each pass the largest remaining element has moved to its final position at the end of the unsorted part, and the process continues until the list is sorted. Its running time is quadratic in general, because up to n passes of up to $n - 1$ comparisons are needed. The optimized version used here stops as soon as a complete pass makes no swap, so an already sorted array is processed in a single pass.

1.2 Insertion Sort

Insertion Sort builds a sorted list one element at a time. Each element is taken from the unsorted portion and inserted into its correct position within the sorted portion, shifting larger elements one place to the right. The amount of work depends strongly on how far elements are from their final positions: an already sorted array needs only about $n - 1$ comparisons, while a reversed array requires the maximum number of shifts.

1.3 Merge Sort

Merge Sort uses a divide-and-conquer strategy. It splits the array into two halves, sorts each half recursively, and then merges the two sorted halves into one sorted sequence. The recursion has $\log n$ levels and each level does linear work, which gives $O(n \log n)$ time in every case. The price is the auxiliary array needed for merging.

1.4 Quick Sort

Quick Sort partitions the list around a pivot element, placing smaller elements on one side and larger elements on the other, and then applies the same procedure recursively to the two parts. When the pivot splits the array into roughly equal parts the running time is $O(n \log n)$, but when the pivot is repeatedly the smallest or largest element the recursion becomes very unbalanced and the running time degrades to $O(n^2)$. The choice of pivot therefore matters a great deal. This report evaluates three pivot strategies:

- **Pivot Choice 1:** the first element is used as the pivot.
- **Pivot Choice 2:** a randomly chosen element is used as the pivot.
- **Pivot Choice 3:** the median of the first, middle, and last elements is used as the pivot.

1.5 Heap Sort

Heap Sort first converts the list into a heap data structure. It then repeatedly extracts the maximum element and places it at the end of the list, which produces ascending order. Both heap construction and the repeated extractions are bounded by $O(n \log n)$, so the comparison-based running time is $O(n \log n)$ for every input order, and the sort needs no additional array.

1.6 Radix Sort

Radix Sort processes the digits of the numbers one position at a time, using a stable internal procedure for each digit. It does not depend on pairwise comparisons between elements, so its running time depends mainly on the number of elements and the number of digits rather than on the order of the data.

2 Data Generation and Experimental Setup

This section describes how the test data were produced and how the performance of the algorithms was measured. The same basic input categories were used for all algorithms.

2.1 Choice of Test Data

To evaluate each algorithm thoroughly, three types of input were used:

- **Sorted in increasing order:** arrays that are already sorted in ascending order.
- **Sorted in decreasing order:** arrays sorted in descending order.
- **Randomly generated elements:** arrays containing randomly generated integers.

The main input sizes recorded in the experimental data are 10,000, 25,000, 50,000, 100,000, 200,000, and 500,000 elements. Bubble Sort and Insertion Sort were recorded only for 10,000 elements, whereas the other algorithms have measurements across all of the larger sizes.

Throughout the report the terms “best case”, “worst case”, and “average case” are used in an empirical sense. For a given algorithm and input size, the *best case* is the smallest measured time among the three input types, the *worst case* is the largest, and the *average case* is the time on random input.

2.2 Experimental Setup

The experiments were implemented in C++. Execution time was measured with the `std::chrono` timing facility and is reported in milliseconds. For each run, the recorded data contain the algorithm name, input type, input size, round number, execution time, and comparison count.

- **Timing unit:** milliseconds.
- **Input sizes:** 10,000 to 500,000 elements where available.
- **Input types:** random, increasing, and decreasing.
- **Algorithms:** Bubble, Insertion, Merge, three Quick Sort versions, Heap, and Radix.

All experiments were run on a single laptop, whose specifications are listed below. The algorithms are implemented as C++ programs that run on the CPU.

Component	Specification
Machine	ASUS TUF Gaming F17 (FX706HF)
Processor	11th Gen Intel Core i5-11400H @ 2.70 GHz
Installed RAM	16 GB (15.7 GB usable), 3200 MT/s
Graphics	NVIDIA GeForce RTX 2050 (4 GB), Intel UHD Graphics
Operating system	Windows 11 Home (version 25H2, OS build 26200.9550), 64-bit

Test machine used for all experiments.

2.3 Stabilizing Measurements

Each experiment was repeated several times to reduce the effect of small timing fluctuations, and every reported value is the average over the recorded rounds.

- **Repetitions:** most combinations of algorithm and input size have five recorded rounds. The first-pivot Quick Sort entries contain two blocks of five rounds in the recorded log, so all ten observations are included in their averages. The 500,000-element decreasing-input entry for Radix Sort has four recorded rounds, and its average uses those four observations.
- **Consistent inputs:** comparisons are made on the same input type and size, so that observed differences can be attributed to the algorithm or the pivot strategy rather than to a change in the experimental category.

3 Comparison of Quick Sort Versions

The performance of Quick Sort depends strongly on the choice of pivot. This section compares the three pivot-selection strategies using the measured execution times:

- Pivot Choice 1: first element,
- Pivot Choice 2: random element,
- Pivot Choice 3: median of the first, middle, and last elements.

The analysis considers the best-case, worst-case, and average-case views of the measurements.

3.1 Best Case Running Time

In the best-case comparison, the time of each Quick Sort version is taken as the smallest of the three input types at each input size. The results show that no single pivot rule is best for every input size and order. The median-of-three version is especially fast on increasing input in this implementation, and the random-pivot version remains consistently strong. The first-pivot version is fast on random input but becomes very expensive when the input is already ordered.

The key point is that pivot selection changes the quality of the partitions. When the pivot produces balanced partitions, the recursive subproblems stay small and the running time follows the expected near- $O(n \log n)$ behavior. When the pivot repeatedly creates highly unbalanced partitions, the number of comparisons grows much faster.

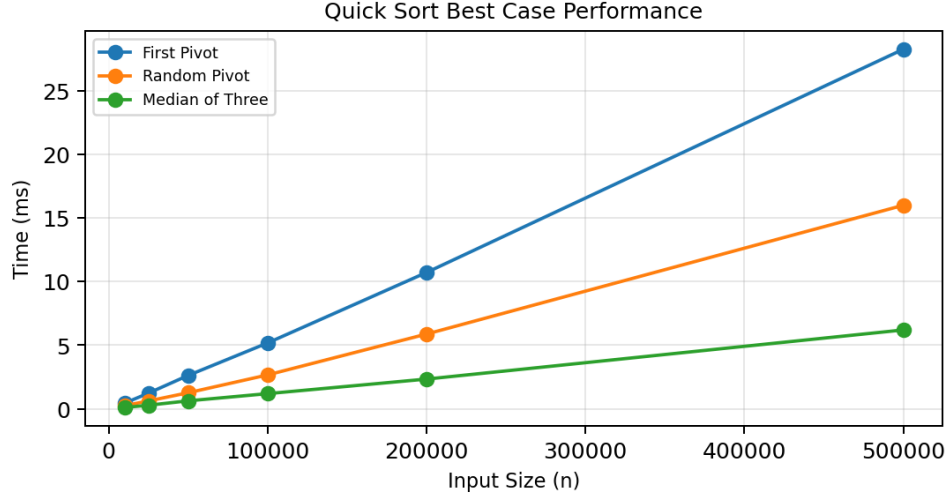


Figure 1: Best Case Running Time of the Three Quick Sort Versions.

3.2 Worst Case Running Time

The worst-case comparison uses the largest measured time among the three input types for each input size. The first-element pivot shows the expected quadratic behavior on ordered data: its average time at 500,000 elements is about 40.569 seconds for increasing input and 61.736 seconds for decreasing input. Compared with its average of 28.2915 ms on random input of the same size, this is a slowdown by a factor of more than two thousand.

A notable experimental result is that the median-of-three implementation also deteriorates strongly on decreasing input. Its average time rises from 6.4291 ms at 10,000 elements to 14,487.6755 ms at 500,000 elements. The input grows by a factor of 50, while the time grows by a factor of more than 2,200, which is close to the factor of 2,500 expected from purely quadratic growth. The report therefore does not assume that a named pivot strategy is automatically safe: the actual implementation and its partition behavior matter. Among the three versions, the random-pivot version is the most stable on the ordered inputs that were tested.

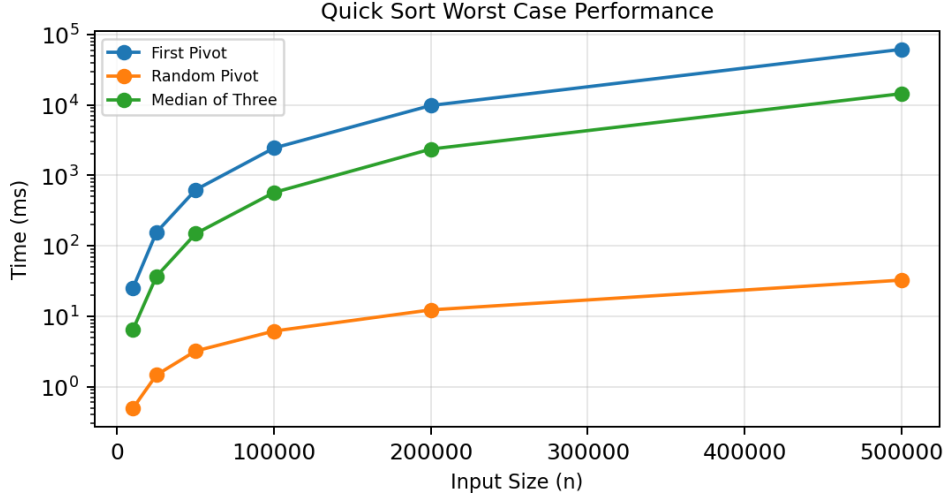


Figure 2: Worst Case Running Time of the Three Quick Sort Versions.

3.3 Average Case Running Time

For the average-case view, randomly generated input is used. On random input all three Quick Sort versions scale far better than in their pathological ordered-input cases. At 500,000 elements the measured averages are 28.2915 ms for the first pivot, 32.5625 ms for the random pivot, and 30.1355 ms for the median-of-three pivot.

The first-pivot version is the fastest on the particular random datasets used in this experiment. This does not contradict the theoretical average-case analysis of Quick Sort, which predicts $O(n \log n)$ time for all three strategies. The first-pivot rule has no extra work for choosing the pivot, while the random and median-of-three rules pay a small constant overhead per partition. The differences among the three are small (about 15% between the fastest and the slowest at 500,000 elements), and practical running time also depends on the actual input, the pivot sequence, implementation constants, and machine conditions.

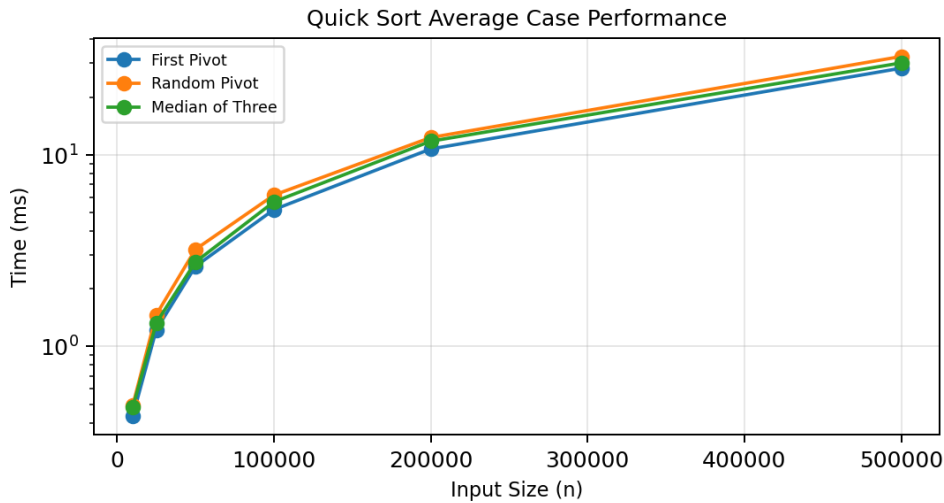


Figure 3: Average Case Running Time of the Three Quick Sort Versions on Random Input.

3.4 Conclusion

The experimental results show that pivot selection has a major effect on Quick Sort. The first-element pivot is very fast on the recorded random data but shows a clear quadratic pattern on increasing and decreasing data. The random-pivot version avoids the large ordered-input blow-up seen in the measurements, because its choice of pivot does not depend on the arrangement of the data. The median-of-three version is extremely fast on increasing input and performs well on random input, but its decreasing-input results show that a particular implementation should not be assumed to be robust for every ordering.

4 Detailed Analysis of Individual Sorting Algorithms

The following subsections analyze each algorithm separately. For each one, performance was studied under increasing, decreasing, and random input wherever the recorded data contain the corresponding measurement.

4.1 Bubble Sort

Bubble Sort shows the expected quadratic behavior when many elements have to be exchanged. In the measurements at 10,000 elements, the increasing input is extremely fast, at about 0.0039 ms, because the optimized implementation terminates after a single pass without swaps. The random and decreasing inputs take about 48.9197 ms and 24.0322 ms respectively.

It is worth noting that random input is slower than decreasing input here, even though a decreasing array requires the largest number of swaps. A plausible explanation is that the comparisons on reversed data are completely predictable for the processor's branch predictor, while on random data they are not, so mispredicted branches add to the cost. Because Bubble Sort was measured at only one input size, Figure 4 shows a single point per input type and cannot show the growth trend.

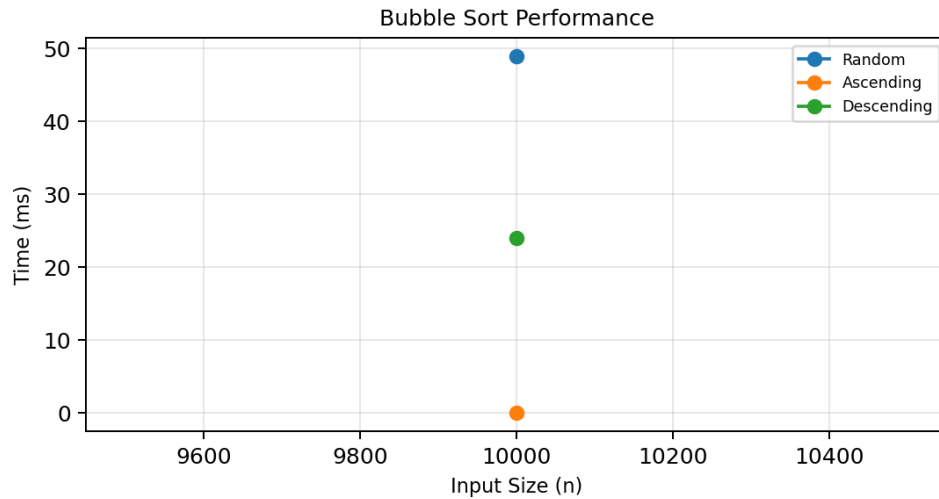


Figure 4: Bubble Sort Performance.

4.2 Insertion Sort

Insertion Sort is efficient on already increasing input because each new element is already in its correct position and no shifting is needed. At 10,000 elements, the measured average is only 0.0164 ms for increasing input, compared with 10.2187 ms for random input and 19.8877 ms for decreasing input. The decreasing case is roughly twice as slow as the random case, which agrees with theory: on random data each element is shifted past about half of the sorted prefix on average, while on reversed data it is shifted past all of it.

Insertion Sort is also noticeably faster than Bubble Sort on random input (10.2187 ms against 48.9197 ms), although both are quadratic. Insertion Sort performs far fewer data movements than Bubble Sort's repeated swaps.

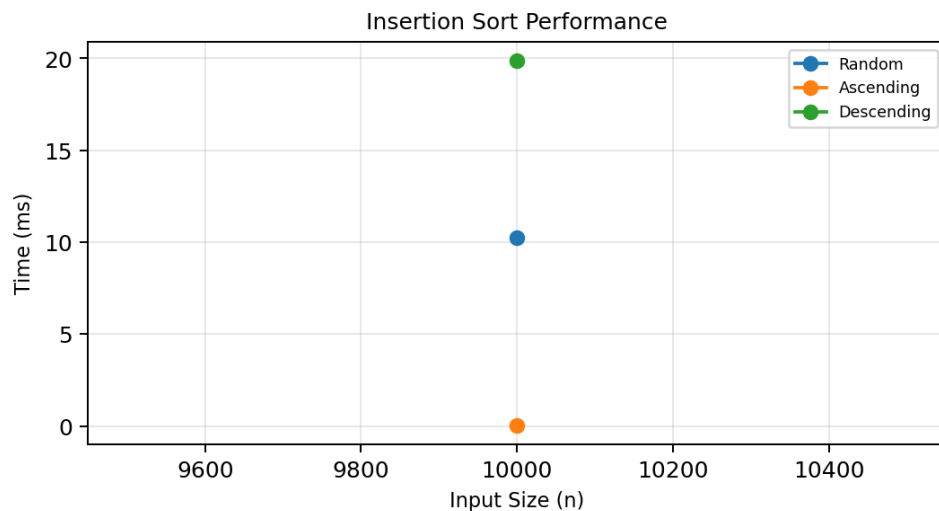


Figure 5: Insertion Sort Performance.

4.3 Merge Sort

Merge Sort consistently shows $O(n \log n)$ -type growth across all the tested input orders. Its measured time varies with the input order because of implementation and constant-factor effects, but the growth remains far more stable than that of the quadratic algorithms. At 500,000 elements, the averages are 67.9383 ms for random input, 41.0334 ms for increasing input, and 45.5028 ms for decreasing input. When the input grows from 10,000 to 500,000 elements (a factor of 50), the random-input time grows from 1.1981 ms to 67.9383 ms, a factor of about 57, which is in line with the factor of about 71 predicted by $n \log n$ growth.

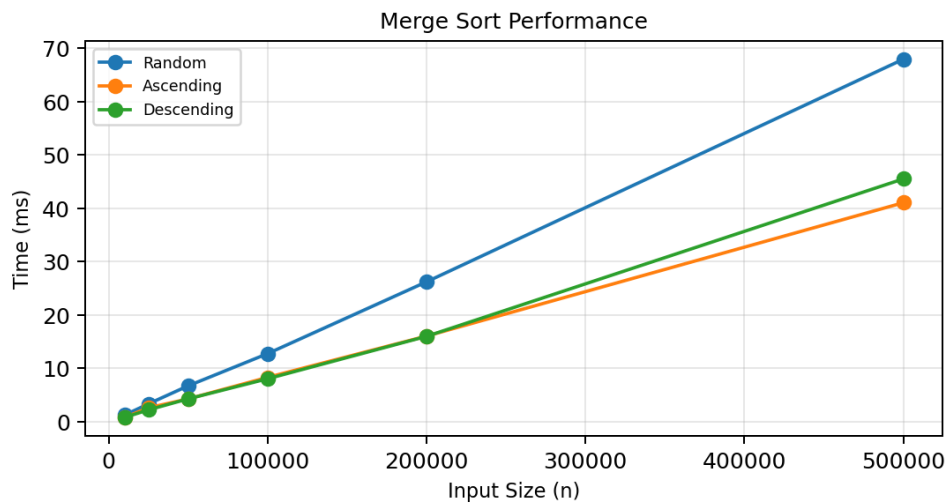


Figure 6: Merge Sort Performance.

4.4 Quick Sort

Quick Sort is highly dependent on the pivot strategy. The first-element pivot is fast on random input but becomes quadratic on ordered input. The random pivot keeps the measured times small for all three input types. Median-of-three is particularly fast on increasing input, but its decreasing-input results grow quadratically, so the implementation exhibits an unfavorable partition pattern for that case. Figure 7 compares the three versions on random input, where they are close to one another and all grow almost linearly on the plotted scale.

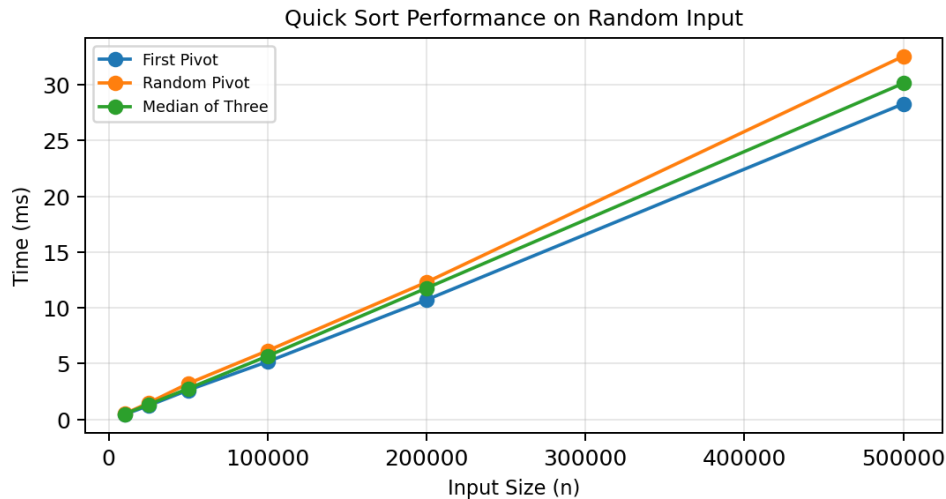


Figure 7: Quick Sort Performance on Random Input.

4.5 Heap Sort

Heap Sort has an $O(n \log n)$ worst-case bound and is not expected to become quadratic because of the input order. The measurements show stable growth. At 500,000 elements the average times are 46.6652 ms for random input, 24.4189 ms for increasing input, and 27.3779 ms for decreasing input. Random input is the slowest case, taking nearly twice as long as the ordered inputs. Heap operations access memory at widely separated positions, and on random data the comparison outcomes are also less predictable, so cache and branch effects are likely contributors. The small crossing of the increasing and decreasing curves at large sizes in Figure 8 is within the range of normal measurement variation.

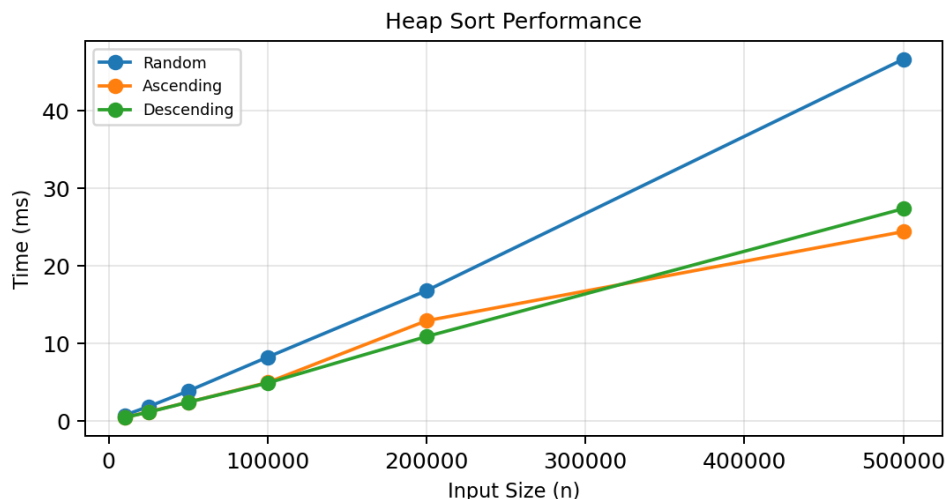


Figure 8: Heap Sort Performance.

4.6 Radix Sort

Radix Sort does not use element-to-element comparisons; in the recorded data its comparison count is zero for every entry. Its measured time is also relatively stable across input orders. At 500,000 elements the average time is 16.9116 ms for random input, 17.0884 ms for increasing input, and 16.2533 ms for decreasing input. Because each digit pass touches every element exactly once regardless of order, this insensitivity to the input is expected. The time grows from 0.2822 ms to 16.9116 ms on random input as the size grows by a factor of 50, which is close to linear scaling.

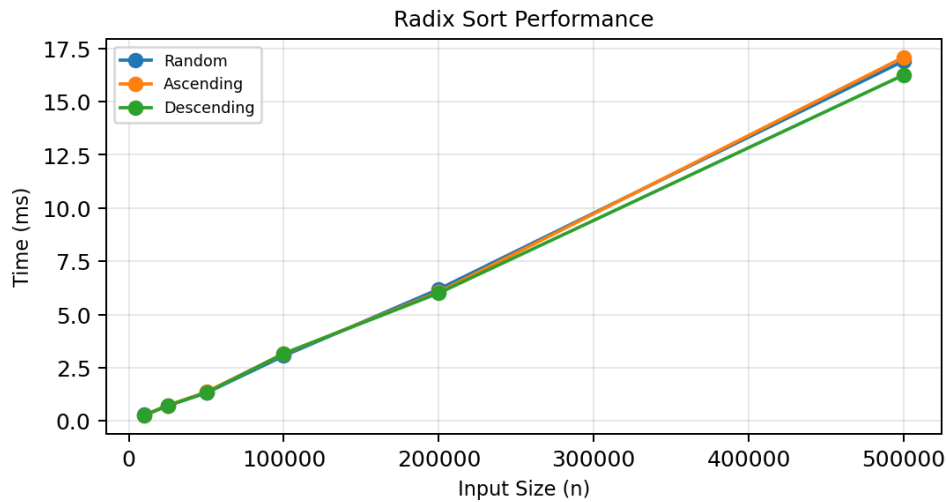


Figure 9: Radix Sort Performance.

5 Comparison of Sorting Algorithms

This section compares the algorithms as a group. The best-case graph uses the smallest measured time among the three input types at each size, the worst-case graph uses the largest measured time, and the average-case graph uses random input.

5.1 Best Case Running Time

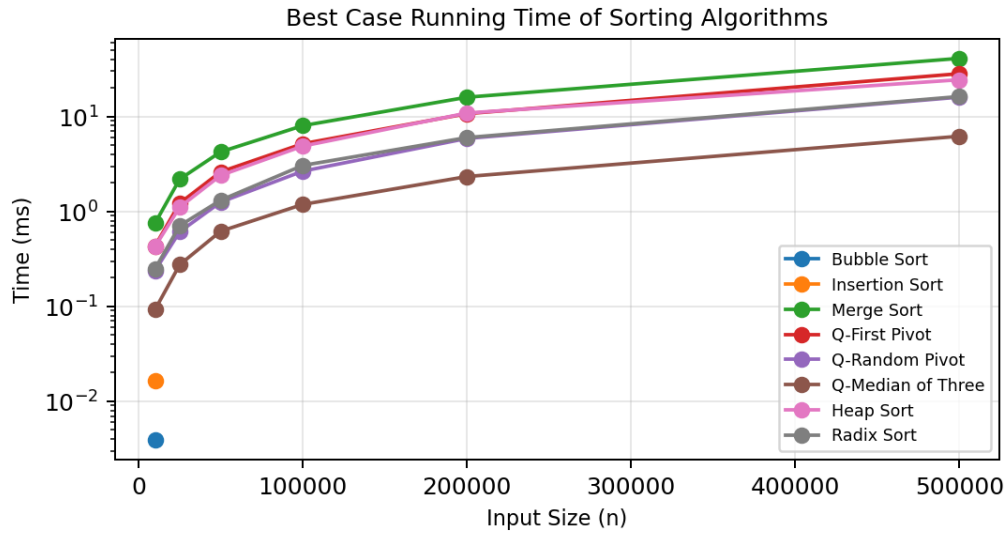


Figure 10: Best Case Running Time of the Sorting Algorithms as a Function of Input Size.

Figure 10 shows why simple algorithms can appear very fast on favorable input. Bubble Sort and Insertion Sort have very low increasing-input times at 10,000 elements, lower than any of the other algorithms at that size. This advantage exists only because the input is already sorted and the implementations stop early or do no shifting. For larger inputs, the efficient algorithms are dominated by their lower growth rates rather than by the favorable constant factors of quadratic methods. At 500,000 elements the lowest best-case time belongs to median-of-three Quick Sort on increasing input (6.1997 ms), followed by Radix Sort (16.2533 ms), Heap Sort (24.4189 ms), and Merge Sort (41.0334 ms).

5.2 Worst Case Running Time

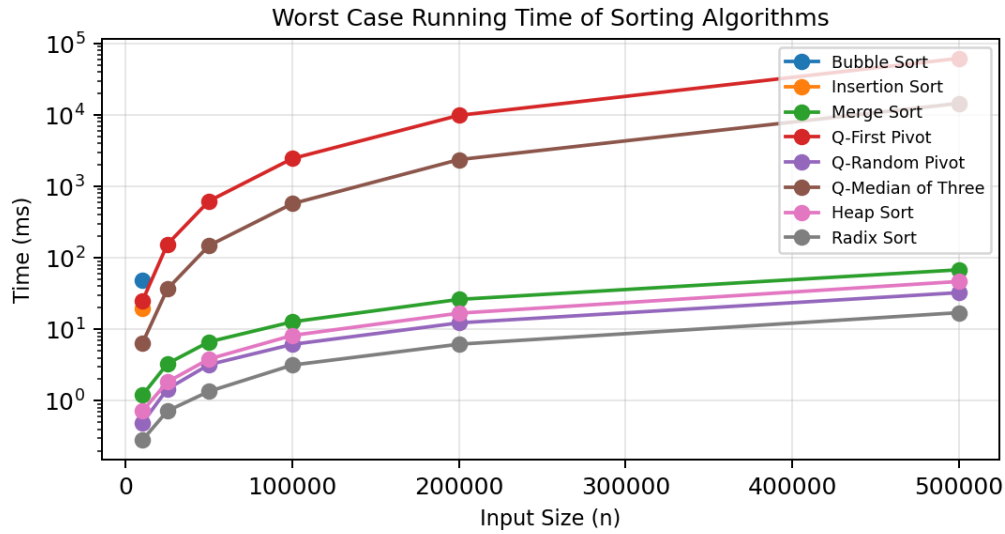


Figure 11: Worst Case Running Time of the Sorting Algorithms as a Function of Input Size.

The worst-case measurements clearly separate the algorithms with quadratic behavior from those with near- $O(n \log n)$ or linear behavior. First-pivot Quick Sort reaches more than 61 seconds at 500,000 decreasing elements, and median-of-three Quick Sort reaches about 14.49 seconds on the same input. By contrast, Merge Sort (67.9383 ms), Heap Sort (46.6652 ms), and Radix Sort (17.0884 ms) stay below 70 ms at that size, and random-pivot Quick Sort also remains in the range of tens of milliseconds. At 10,000 elements, the worst case of Bubble Sort (48.9197 ms) is already about forty times that of Merge Sort (1.1981 ms), which shows how quickly the quadratic methods become impractical.

5.3 Average Case Running Time

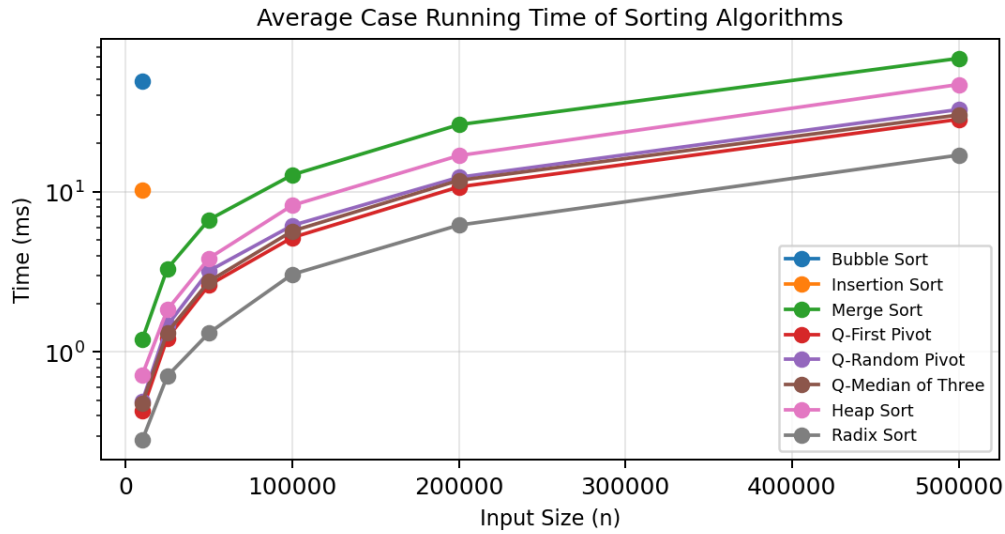


Figure 12: Average Case Running Time of the Sorting Algorithms on Random Input.

For randomly generated input, Radix Sort is the fastest algorithm at the larger sizes in the measurements. At 500,000 elements its average is 16.9116 ms. First-pivot Quick Sort is next at 28.2915 ms, followed by median-of-three at 30.1355 ms and random-pivot Quick Sort at 32.5625 ms. Heap Sort and Merge Sort are slower, at 46.6652 ms and 67.9383 ms respectively. Bubble Sort and Insertion Sort are not available at 500,000 elements in the data.

5.4 Conclusion

The experimental analysis shows that there is no single best sorting algorithm for every situation. The input order, the pivot selection, and implementation details all affect the measured time.

- **Best-case performance:** Bubble Sort and Insertion Sort can be extremely fast on already sorted input when the implementation stops early.
- **Worst-case performance:** First-pivot Quick Sort shows the clearest quadratic growth on ordered input. The median-of-three implementation used here also shows quadratic-like deterioration on decreasing input.
- **Average-case performance:** On random data, Radix Sort is the fastest at the larger sizes, while the Quick Sort versions also scale well.
- **Reliable choices:** Merge Sort and Heap Sort maintain predictable $O(n \log n)$ -type growth for every input order, and random-pivot Quick Sort provides strong protection against the ordered-input behavior seen with first-pivot Quick Sort.

In summary, the experimental evidence supports choosing an algorithm according to the expected data distribution, rather than relying only on its asymptotic label.

6 Correlation of Comparisons with Execution Time

Besides time, the number of comparisons made by each algorithm was recorded. This section examines how well the comparison count explains the running time. Where the correlation between the two is close to 1, counting comparisons is a good machine-independent predictor of speed. Where it is lower, other costs such as memory access, data movement, and branching make a larger contribution.

6.1 Insertion Sort

In the best case, Insertion Sort performs approximately $n - 1$ comparisons. The measured time per comparison remains nearly constant, at roughly 8.2×10^{-7} ms, and the correlation between time and comparison count across the measurements is approximately 0.9998. This is a strong experimental match between comparison count and execution time.

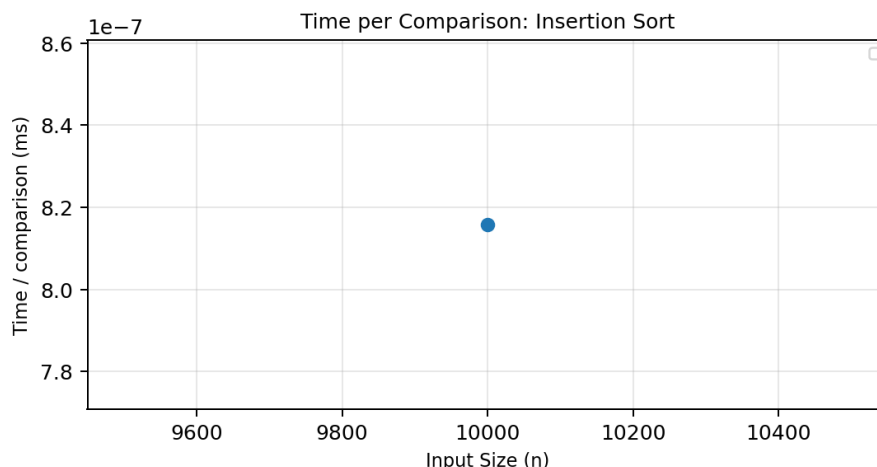


Figure 13: Time per Comparison for Insertion Sort.

6.2 Bubble Sort (Optimized)

Bubble Sort also has a best-case comparison count close to $n - 1$ when the input is already sorted. Its time per comparison is small, at roughly 6.2×10^{-7} ms, and the early-termination optimization prevents unnecessary passes. The measured correlation between time and comparisons is about 0.8609 across the Bubble Sort measurements, which is clearly positive but weaker than for Insertion Sort. Since Bubble Sort also spends time on swaps, the comparison count alone explains less of its running time.

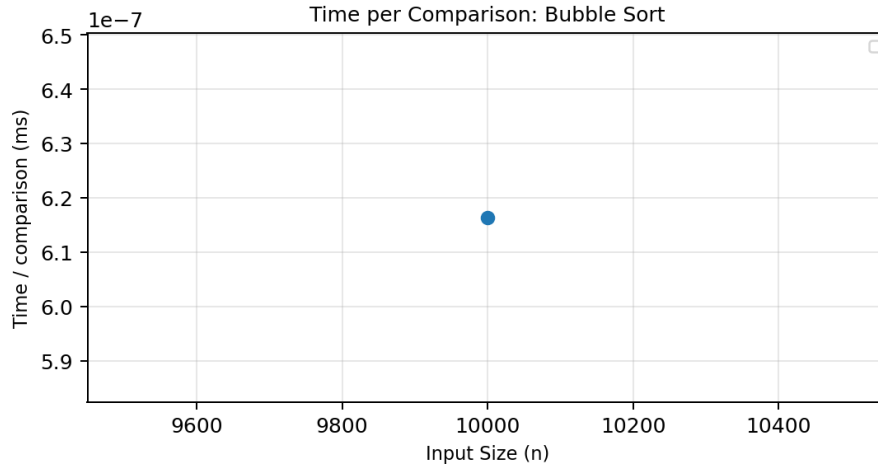


Figure 14: Time per Comparison for Bubble Sort.

6.3 Heap Sort

Heap Sort performs many comparisons during heap construction and during the repeated extractions. The measured correlation between time and comparisons is about 0.9348. Small variations are expected because execution time also includes memory accesses, swaps, branching, and system-level effects. As Figure 15 shows, the time per comparison stays within a narrow band of roughly 1.9×10^{-6} to 2.25×10^{-6} ms over the whole range of sizes.

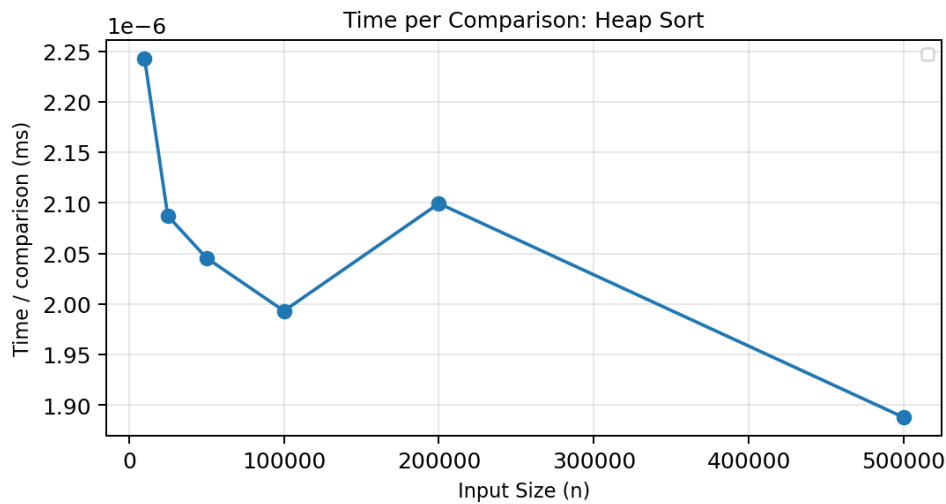


Figure 15: Time per Comparison for Heap Sort.

6.4 Merge Sort

Merge Sort makes a number of comparisons proportional to $n \log n$. The measured correlation between time and comparisons is about 0.9941, showing a very strong relationship. The small deviations arise from memory access and the cost of moving elements in addi-

tion to the comparisons themselves. The time per comparison is highest at the smaller sizes and settles to a nearly constant value at the larger sizes, where fixed overheads are spread over more work.

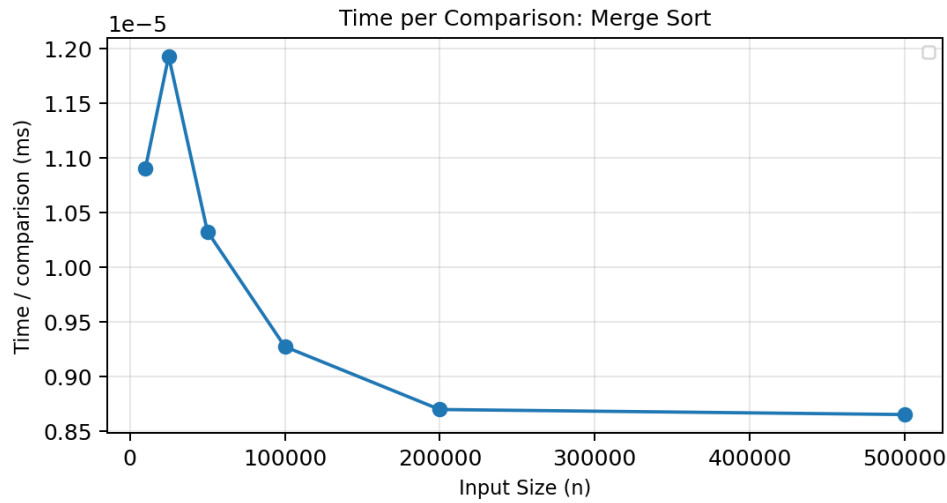


Figure 16: Time per Comparison for Merge Sort.

6.5 Quick Sort (Median-of-Three Pivot)

The median-of-three measurements show an almost perfect correlation between execution time and comparison count, approximately 0.99999. This is expected because the implementation performs a very large number of comparisons whenever its partitioning becomes unfavorable, and those comparisons then dominate the running time. The time per comparison actually falls slightly as the input grows, from about 1.65×10^{-6} ms to about 1.41×10^{-6} ms. The result is a useful example of why counting comparisons helps to explain observed running time.

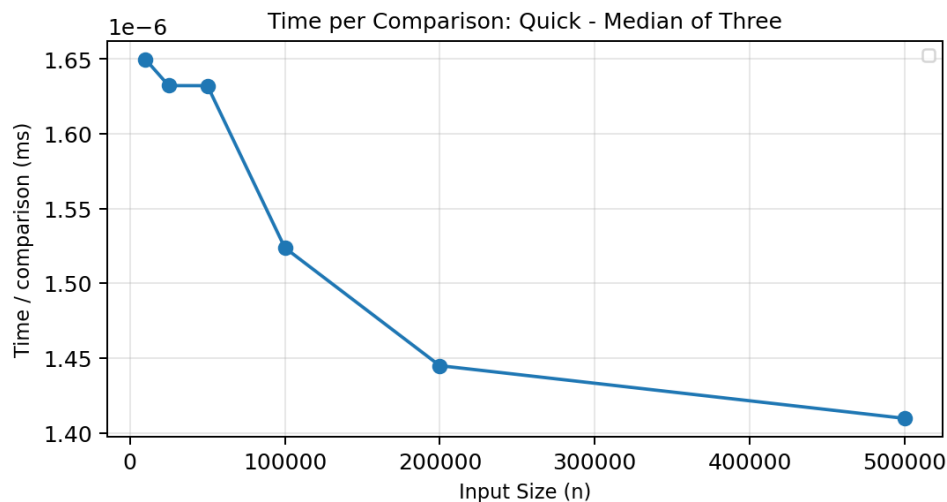


Figure 17: Time per Comparison for Quick Sort with Median-of-Three Pivot.

6.6 Radix Sort

Radix Sort does not rely on comparisons. Its comparison count is recorded as zero in the data, so time per comparison is not a meaningful metric. Instead, its running time depends mainly on the number of elements and the number of digit passes, and Figure 18 shows that it grows almost linearly with the input size for all three input orders.



Figure 18: Radix Sort Execution Time; Comparison Count is Zero.

6.7 Conclusion

For the comparison-based algorithms (Insertion, Bubble, Heap, Merge, and Quick Sort), execution time generally follows the number of comparisons, as the strong correlations above show. For small inputs, timing fluctuations can be caused by system overhead and measurement resolution. For larger inputs, cache behavior, memory access, branch behavior, and implementation details also affect the ratio between time and comparisons, which is why the time per comparison differs from one algorithm to another and why the correlation is not exactly 1. In contrast, Radix Sort does not use comparisons: its measured comparison count is zero, while its execution time remains approximately linear in the input size for the tested integer data.

7 Execution Time Analysis

The following tables present the average execution time in milliseconds for random, increasing, and decreasing inputs. The values are averages over the recorded rounds in the experimental data.

7.1 Bubble Sort

Input Size	Random	Increasing	Decreasing
10000	48.9197	0.0039	24.0322

Table 1: Average Execution Time for Bubble Sort

7.2 Insertion Sort

Input Size	Random	Increasing	Decreasing
10000	10.2187	0.0164	19.8877

Table 2: Average Execution Time for Insertion Sort

Note: Bubble Sort and Insertion Sort have only the 10,000-element measurements in the recorded results. No larger values have been estimated or invented.

7.3 Merge Sort

Input Size	Random	Increasing	Decreasing
10000	1.1981	0.7517	0.7674
25000	3.3076	2.5725	2.1867
50000	6.6892	4.2457	4.2464
100000	12.7277	8.2835	8.0188
200000	26.1980	16.0209	15.9771
500000	67.9383	41.0334	45.5028

Table 3: Average Execution Time for Merge Sort

7.4 Heap Sort

Input Size	Random	Increasing	Decreasing
10000	0.7188	0.4263	0.4383
25000	1.8355	1.1562	1.1106
50000	3.8448	2.4047	2.4005
100000	8.2178	4.9474	4.8840
200000	16.8048	12.9187	10.8805
500000	46.6652	24.4189	27.3779

Table 4: Average Execution Time for Heap Sort

7.5 Radix Sort

Input Size	Random	Increasing	Decreasing
10000	0.2822	0.2674	0.2489
25000	0.7076	0.7214	0.7155
50000	1.3140	1.3555	1.3251
100000	3.0536	3.1490	3.1611
200000	6.1864	6.0532	6.0031
500000	16.9116	17.0884	16.2533

Table 5: Average Execution Time for Radix Sort

7.6 Quick Sort (Median-of-Three Pivot)

For Quick Sort, the table below uses the median-of-three version as the representative Quick Sort table, while Section 3 compares all three pivot strategies separately. The values are the actual measured averages and are not substituted from theory.

Input Size	Random	Increasing	Decreasing
10000	0.4806	0.0923	6.4291
25000	1.3248	0.2745	36.7874
50000	2.7569	0.6190	148.6994
100000	5.6769	1.1861	572.3029
200000	11.7684	2.3322	2366.2865
500000	30.1355	6.1997	14487.6755

Table 6: Average Execution Time for Quick Sort (Median-of-Three Pivot)

7.7 Quick Sort: Comparison of the Three Versions

The following compact table summarizes the three Quick Sort versions on random input. It allows a direct comparison of the measured average running time at every available input size.

Input Size	First Pivot	Random Pivot	Median of Three
10000	0.4326	0.4913	0.4806
25000	1.2204	1.4631	1.3248
50000	2.6159	3.2077	2.7569
100000	5.1724	6.1574	5.6769
200000	10.7206	12.2993	11.7684
500000	28.2915	32.5625	30.1355

Table 7: Average Random-Input Execution Time of Quick Sort Versions (ms)

The final comparison shows the main practical result of the experiment. For random input, first-pivot Quick Sort has the lowest measured time of the three at every listed size. However, the ordered-input experiments show that it is not robust to adverse input order. The random pivot gives the most stable behavior across input types, while median-of-three is excellent on increasing and random data but shows a large deterioration on decreasing data in this implementation.

Overall Conclusion

This study compared six sorting algorithms, including three Quick Sort variants, on random, increasing, and decreasing integer arrays of up to 500,000 elements. The main findings are as follows.

1. **Asymptotic notation is a guide, not a verdict.** The quadratic algorithms are unusable at scale, but among the $O(n \log n)$ and linear methods the constant factors, memory behavior, and input order decided the ranking.
2. **Pivot selection is decisive for Quick Sort.** First-pivot Quick Sort ranges from 28.2915 ms on random input to 61.736 seconds on decreasing input at 500,000 elements. The random pivot is the most dependable of the three variants.
3. **A named strategy is not a guarantee.** The median-of-three implementation was excellent on increasing and random data, but grew quadratically on decreasing data, which suggests that its partitioning step should be inspected and corrected before it is relied upon.
4. **Merge Sort and Heap Sort are predictable.** Their running times are largely insensitive to the input order and grow in line with $n \log n$.
5. **Radix Sort is fastest on the tested random integers.** It needs no comparisons and scales almost linearly, at the cost of being limited to data that can be split into digits.
6. **Comparison counts explain time well for comparison sorts.** The measured correlations range from 0.8609 to 0.99999, with the remaining variation attributable to memory access, data movement, and branching.

Limitations and Future Work

The conclusions should be read with the following limitations in mind. Bubble Sort and Insertion Sort were measured only at 10,000 elements, so their growth trends could not be examined. Each result is an average of about five rounds and reflects a single implementation and the single machine described in Section 2.2. Only integer data and one random distribution were used. Future work could extend the quadratic algorithms to larger sizes, test further input patterns such as nearly sorted data and data with many

duplicate values, examine the cause of the decreasing-input behavior of the median-of-three implementation, and compare the algorithms on additional hardware and compilers.

End of Report